

List of Lab works

La b no	Title/ Question	Submission Date	Signature	Remarks
1	WAP to illustrate Caesar Cipher.	2026/04/13		
2	WAP to illustrate Playfair cipher.	2026/04/13		
3	WAP to illustrate Rail Fence cipher.	2026/04/13		
4	WAP to simulate modular arithmetic operation.	2026/04/13		
5	WAP to implement vigenere and vernam cipher.	2026/04/13		
6	WAP to implement Hill cipher.	2026/04/13		
7	WAP to find whether the given positive integer is prime or composite using Lucas Lehmer test.	2026/04/13		
8	WAP to find the GCD of given two positive integers using Euclidean algorithm.	2026/04/13		
9	Use the program [5] to find whether the given two integers are co-prime or not.	2026/04/13		
10.	Illustrate the Euler totient function.	2026/04/13		
11.	WAP that takes two integers as parameter and find out whether the first parameter is primitive root of second parameter or not.	2026/04/13		
12.	WAPto show how RSA generates the private and public key.	2026/04/13		
13.	Write a program to illustrate the Diffie Hellman Key exchange protocol.	2026/04/13		
14.	WAP to find the multiplicative inverse using extended Euclidean algorithm.	2026/04/13		
15.	WAP to implement the Elgamal cryptosystem.	2026/04/13		

1. Caesar Cipher Implementation

C-Program:

```
#include<stdio.h>
#include<string.h>
#include"helper."

static char encrypt(char ch, int key) {
    if (ch >= 'A' && ch <= 'Z') return (char)((ch - 'A' + key) % 26 + 'A');
    if (ch >= 'a' && ch <= 'z') return (char)((ch - 'a' + key) % 26 + 'a'); return ch; }

int main(void) {
    string plain = get_string("Enter the plain text: ");
    int key = ((get_int("Enter the key: ") % 26) + 26) % 26;
    int len = strlen(plain);
    string cipher = (string)malloc(len + 1);
    for (int i = 0; i < len; i++) cipher[i] = encrypt(plain[i], key);
    cipher[len] = '\0';
    printf("Plain Text: %s\n", plain);
    printf("Cipher Text: %s\n", cipher);
    print_details();
    free(cipher);
    printf("Lab No:1 by Raman Thapa, Roll No:88, Section: C");
    return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc CaesarCipher.c -o Caesar
Cipher } ; if ($?) { .\CaesarCipher }
Enter the plain text: MEET ME AFTER THE TOGA PARTY
Enter the key: 3
Plain Text: MEET ME AFTER THE TOGA PARTY
Cipher Text: PHHW PH DIWHU WKH WRJD SDUWB

Name: Raman Thapa, Roll No:88/080, Section:C
```

2. Playfair Encryption

C-Program:

```
#include<stdio.h>
#include<string.>
#include<ctype.>
#include"helper.h"
" #define MX 5
int main(void) {
char matrix[MX][MX];
string key_in = get_string("Enter Key: ");
string plain = get_string("Enter Plaintext:
"); if (!key_in || !plain) return 1;
int used[26] = {0};
used['J' - 'A'] = 1;
int row = 0, col = 0;
string sources[] = {key_in, "ABCDEFGHIJKLMNOPQRSTUVWXYZ"};
for (int s = 0; s < 2; s++) {
for (int i = 0; sources[s][i] != '\0'; i++)
{ if (!isalpha(sources[s][i]))
continue; char c =
toupper(sources[s][i]);
if (c == 'J') c = 'I';
if (!used[c - 'A'])
{ matrix[row][col++] = c;
used[c - 'A'] = 1;
if (col == MX) { col = 0; row++; } } }
printf("\nKey Matrix:\n");
for (int i = 0; i < MX; i++) {
for (int j = 0; j < MX; j++) printf("%c ", matrix[i][j]);
printf("\n"); }
int len = strlen(plain);
string cipher = malloc(len * 2 +
1); int p_idx = 0, c_idx = 0;
while (plain[p_idx] != '\0') {
if (!isalpha(plain[p_idx])) { p_idx++; continue;
} char a = toupper(plain[p_idx++]);
if (a == 'J') a = 'I';
char b = 'X';
int next = p_idx;
```

```

while (plain[next] != '\0' && !isalpha(plain[next])) next++;
if (plain[next] != '\0' && toupper(plain[next]) != (a == 'I' ? 'J' :
a)) { b = toupper(plain[next]);
if (b == 'J') b = 'I'; p_idx = next + 1; } int r1, c1, r2, c2;
for (int i = 0; i < 25; i++) {
if (matrix[i/5][i%5] == a) { r1 = i/5; c1 = i%5; }
if (matrix[i/5][i%5] == b) { r2 = i/5; c2 = i%5; }
} if (r1 == r2) {
cipher[c_idx++] = matrix[r1][(c1 + 1) % MX];
cipher[c_idx++] = matrix[r2][(c2 + 1) % MX];
} else if (c1 == c2) {
cipher[c_idx++] = matrix[(r1 + 1) % MX][c1];
cipher[c_idx++] = matrix[(r2 + 1) % MX][c2];
} else {
cipher[c_idx++] = matrix[r1][c2];
cipher[c_idx++] = matrix[r2][c1]; } }
cipher[c_idx] = '\0';
printf("\nPlaintext: %s\nCiphertext: %s\n", plain, cipher);
print_details();
free(cipher);
return 0; }

```

Output:

```

Name: Raman Thapa, Roll No: 88/080, Section: C
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc PlayfairCipher.c -o PlayfairCipher } ; if ($?) { .\PlayfairCipher }
Enter Key: MONARCHY
Enter Plaintext: THE IMITATION GAME

Key Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Plaintext: THE IMITATION GAME
Ciphertext: PDFKAESRSKNAINCL

Name: Raman Thapa, Roll No: 88/080, Section: C

```

3. Rail fence Cipher

C -Program:

```
#include<stdio.h>
#include<string.h>
#include"helper."
int main(void) {
    string plain = get_string("Enter the plain text:
"); if (!plain) return 1;
    int len = strlen(plain);
    string cipher = (string)malloc(len +
1); if (!cipher) return 1;
    int rail = get_int("Enter the number of rails:
"); char matrix[rail][len];
    for (int i = 0; i < rail; i++)
        for (int j = 0; j < len; j++) matrix[i][j] = '\n';
    int dir_down = 0, row = 0, col = 0;
    for (int i = 0; i < len; i++) {
        if (row == 0 || row == rail - 1) dir_down = !dir_down;
        matrix[row][col++] = plain[i];
        row += dir_down ? 1 : -1;
    }
    int idx = 0;
    for (int i = 0; i < rail; i++)
        for (int j = 0; j < len; j++)
            if (matrix[i][j] != '\n')
                cipher[idx++] = matrix[i][j];
    cipher[idx] = '\0';
    printf("Plain Text: %s\n", plain);
    printf("Cipher Text: %s\n", cipher);
    print_details();
    free(cipher);
    return 0; }
```

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc RailfenceCipher.c
-o RailfenceCipher } ; if ($?) { .\RailfenceCipher }
Enter the plain text: YELLOW SUBMARINE
Enter the number of rails: 3
Plain Text: YELLOW SUBMARINE
Cipher Text: YOURELWSBAIEL MN

Name: Raman Thapa, Roll No:88/080, Section:C
```

Output:

4. Modular Arithmetic Operation:

C Program:

```
#include
<stdio.h>
#include "helper.
h" int main(void){
int a = get_int("Enter the first number: ");
int b = get_int("Enter the second number:
"); int mod = get_int("Enter the modulus:
"); if(mod <= 0){
printf("Modulus must be a positive integer.\n"); return 1; }
int sum = (a % mod + b % mod) % mod;
int sub = (a % mod - b % mod + mod) %
mod; int mul = (a % mod * b % mod) %
mod;
printf("The result of (%d + %d) mod %d is: %d\n", a, b, mod, sum);
printf("The result of (%d * %d) mod %d is: %d\n", a, b, mod, mul);
printf("The result of (%d - %d) mod %d is: %d\n", a, b, mod, sub);
print_details();
return 0; }
```

Output:

```
> PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc ModularArithmetic.c
-o ModularArithmetic } ; if ($?) { .\ModularArithmetic }
Enter the first number: 524
Enter the second number: 68
Enter the modulus: 12
The result of (524 + 68) mod 12 is: 4
The result of (524 * 68) mod 12 is: 4
The result of (524 - 68) mod 12 is: 0

Name: Raman Thapa, Roll No:88/080, Section:C
```

5. Vigenere Cipher and Vernam Cipher

C Program:

```
#include<stdio.h>
#include<string.h>
#include"helper."
void vigenere_cipher(string text, string key, char
result[]) { int text_len = strlen(text);
int key_len = strlen(key);
for (int i = 0; i < text_len; i++) {
if (text[i] >= 'A' && text[i] <= 'Z') {
result[i] = ((text[i] - 'A') + (key[i % key_len] - 'A')) % 26 + 'A'; }
else if (text[i] >= 'a' && text[i] <= 'z') {
result[i] = ((text[i] - 'a') + (key[i % key_len] - 'a')) % 26 + 'a'; }
else {
result[i] = text[i];
}
}
result[text_len] = '\0'; }
void vernam_cipher(string text, string key, char
result[]) { int text_len = strlen(text);
for (int i = 0; i < text_len; i++) {
result[i] = text[i] ^ key[i % strlen(key)]; }
result[text_len] = '\0'; }
int main(void) {
string text = get_string("Enter the text: ");
string v_key = get_string("Enter Vigenere key:
"); int text_len = strlen(text);
char result[text_len + 1];
vigenere_cipher(text, v_key, result);
printf("Vigenere Cipher Result: %s\n",
result);
printf("\nVernam Cipher requires a key of length %d.\n", text_len);
string vernam_key;
do {
vernam_key = get_string("Enter Vernam key:
"); if (strlen(vernam_key) != text_len)
{ printf("Invalid length! Try again.\n");
}
}
while (strlen(vernam_key) != text_len);
```

```

vernam_cipher(text, vernam_key,
result);
printf("Vernam Cipher Result (Hex): ");
for (int i = 0; i < text_len; i++)
{ printf("%02X ", (unsigned
char)result[i]); }print_details(); return 0;
}

```

Output:

```

● PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc VigenereVernamCipher.c -o V
nereVernamCipher } ; if ($?) { .\VigenereVernamCipher }
Enter the text: JonSnow
Enter Vigenere key: TheMadKing
Vigenere Cipher Result: CvrEnra

Vernam Cipher requires a key of length 7.
Enter Vernam key: Westeros
Invalid length! Try again.
Enter Vernam key: Westero
Vernam Cipher Result (Hex): 1D 0A 1D 27 0B 1D 18
Name: Raman Thapa, Roll No:88/080, Section:C

```


6. Hill Cipher

C Program:

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#include"helper."

void hillCipher(string text, int key[2][2]);
static int mod26(int x){
int r = x % 26;
return (r < 0) ? r + 26 : r; }
int main(void) {
string text = get_string("Enter the text to
encrypt: "); int key[2][2];
printf("Enter the 2x2 key matrix (4 integers):\n");
for (int i = 0; i < 2; i++){
for (int j = 0; j < 2; j++){
if (scanf("%d", &key[i][j]) != 1)
{ printf("Invalid matrix
input.\n"); return 1; } } }
int det = key[0][0] * key[1][1] - key[0][1] * key[1][0];
det = mod26(det);
if (det == 0 || det % 2 == 0 || det % 13 == 0)
printf("Warning: This key matrix is not invertible and cannot be decrypted!\n");
hillCipher(text, key);
print_details();
return 0; }

void hillCipher(string text, int key[2][2]){
int len = strlen(text);
char clean_text[len + 1]; int clean_len = 0;
for (int i = 0; i < len; i++) {
if (isalpha((unsigned char)text[i])) {
clean_text[clean_len++] = (char)toupper((unsigned char)text[i]); }
} clean_text[clean_len] = '\0';
if (clean_len == 0){
printf("No alphabetic text to encrypt.\n"); return; }
if (clean_len % 2 != 0)
{ clean_text[clean_len++] = 'X';
```

```

clean_text[clean_len] = '\0'; }
char encrypted[clean_len + 1];
for (int i = 0; i < clean_len; i += 2)
{ int x1 = clean_text[i] - 'A'; int x2
= clean_text[i + 1] - 'A';
encrypted[i] = mod26(key[0][0] * x1 + key[0][1] * x2) + 'A';
encrypted[i + 1] = mod26(key[1][0] * x1 + key[1][1] * x2) + 'A';
} encrypted[clean_len] = '\0';
printf("Encrypted text: %s\n", encrypted); }

```

Output:

```

PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc HillCipher.c -o HillCipher
if ($?) { .\HillCipher }
Enter the text to encrypt: WhiteNights
Enter the 2x2 key matrix (4 integers):
3 2
4 3
Encrypted text: CFKLMDKYHHWL

Name: Raman Thapa, Roll No: 88/080, Section:C

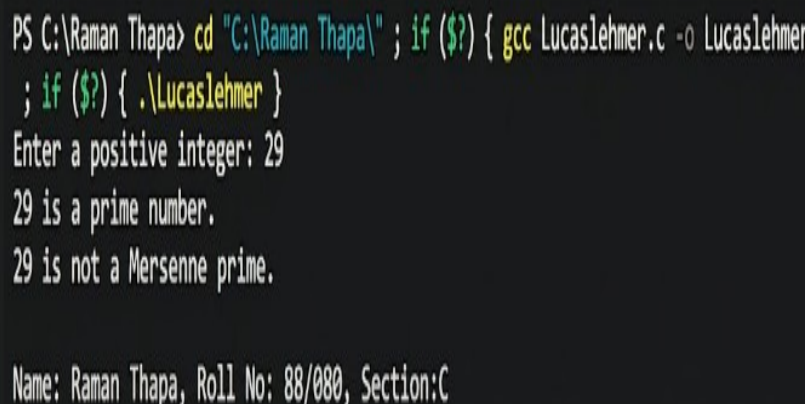
```

7. Prime number check using Lucas Lehmer test.

C Program:

```
#include <stdio.h>
#include <math.h>
#include <stdbool.h>
#include "helper.h"
typedef long long ll;
bool is_prime(int n) {
    if (n <= 1) return
    false; if (n <= 3)
    return true;
    if (n % 2 == 0 || n % 3 == 0) return
    false; for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    } return true; }
bool lucas_lehmer_test(int p) {
    if (p == 2) return true; // 2 is
    prime ll s = 4;
    ll m = (1LL << p) - 1;
    for (int i = 0; i < p - 2; i++)
        { s = (s * s - 2) % m; }
    return s == 0; }
int main() {
    int n = get_int("Enter a positive integer:
    "); if (n < 1) {
        printf("Not a positive integer.\n");
        return 1; }
    if (is_prime(n)) {
        printf("%d is a prime number.\n",
        n); if (lucas_lehmer_test(n)) {
            printf("%d is a Mersenne prime.\n", n);
        } else {
            printf("%d is not a Mersenne prime.\n", n); }
        } else {
            printf("%d is a composite number.\n", n); }
        print_details();
        return 0; }
```

Output:



```
PS C:\Raman Thapa> cd "C:\Raman Thapa\" ; if ($?) { gcc LucasLehmer.c -o LucasLehmer
; if ($?) { .\LucasLehmer }
Enter a positive integer: 29
29 is a prime number.
29 is not a Mersenne prime.

Name: Raman Thapa, Roll No: 88/080, Section:C
```

8. GCD by Euclidean Algorithm:

C Program:

```
#include
<stdio.h>
#include
"helper.h"      int
main() {
int a = get_int("Enter the first positive integer: ");
int b = get_int("Enter the second positive integer:
"); if (a <= 0 || b <= 0) {
printf("Please enter positive integers only.\n
n"); return 1; }
int result = gcd(a, b);
printf("The GCD of %d and %d is: %d\n", a, b, result);
print_details();
return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc GCD_eu
idean.c -o GCD_euclidean } ; if ($?) { .\GCD_euclidean }
Enter the first positive integer: 522
Enter the second positive integer: 42
The GCD of 522 and 42 is: 6

Name: Raman Thapa, Roll No: 88/080, Section:C
```

9. Co-Prime check

C Program:

```
#include
<stdio.h>
#include
"helper.h"      int
main() {
int a = get_int("Enter the first positive integer: ");
int b = get_int("Enter the second positive integer:
"); if (a <= 0 || b <= 0) {
printf("Please enter positive integers only.\n
n"); return 1; }
int result = gcd(a,
b); if (result == 1) {
printf("%d and %d are co-prime.\n", a, b);
} else {
printf("%d and %d are not co-prime. GCD is %d.\n", a, b, result); }
print_details();
return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\\" ; if ($?) { gcc Copri
heck.c -o Coprimecheck } ; if ($?) { .\Coprimecheck }
Enter the first positive integer: 7
Enter the second positive integer: 21
7 and 21 are not co-prime. GCD is 7.

Name: Raman Thapa, Roll No: 88/080, Section:C
PS C:\Raman Thapa> cd "c:\Raman Thapa\\" ; if ($?) { gcc Copri
heck.c -o Coprimecheck } ; if ($?) { .\Coprimecheck }
Enter the first positive integer: 16
Enter the second positive integer: 15
16 and 15 are co-prime.

Name: Raman Thapa, Roll No: 88/080, Section:C
```

10. Euler Totient Function

C Program:

```
#include
<stdio.h>
#include
"helper.h"
int euler_totient(int n)
{ int count = 0;
  for (int i = 1; i < n; i++)
  { if (gcd(i, n) == 1)
    { count++; }
  }
  return count;
}
int main() {
  int n = get_int("Enter a positive integer:
"); if (n <= 0) {
  printf("Please enter a positive integer.\n
n"); return 1; }
  int result = euler_totient(n);
  printf("The Euler totient function for %d is: %d\n", n, result);
  print_details();
  return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc EulerT
● ient.c -o EulerTotient } ; if ($?) { .\EulerTotient }
Enter a positive integer: 55
The Euler totient function for 55 is: 40

Name: Raman Thapa, Roll No: 88/080, Section:C
```

11. Primitive root or not?

C Program:

```
#include
<stdio.h>
#include
"helper.h"
int is_primitive_root(int a, int
p) { if (gcd(a, p) != 1) return 0;
int order = 1, value = a %
p; while (value != 1) {
value = (value * a) % p;
if (++order > p - 1) return 0; }
return order == p - 1; }
int main() {
int a = get_int("Enter the first positive integer (potential primitive
root): "); int p = get_int("Enter the second positive integer (modulus):
");
if (is_primitive_root(a, p)) {
printf("%d is a primitive root modulo %d.\n", a, p);
} else {
printf("%d is not a primitive root modulo %d.\n", a, p);
}
print_details();
return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc Primitiveroot.c -o Primitiv
ot } ; if ($?) { .\Primitiveroot }
Enter the first positive integer (potential primitive root): 3
Enter the second positive integer (modulus): 5
3 is a primitive root modulo 5.
Name: Raman Thapa, Roll No: 88/080, Section:C
```

12. RSA Algorithm for public/private key generation.

C Program:

```
#include <bits/stdc++.h> using namespace
std;
int power(int base, int expo, int
m) { int res = 1;
base = base % m;
while (expo > 0) {
if (expo & 1)
res = (res * 1LL * base) % m;
base = (base * 1LL * base) %
m; expo = expo / 2;
}
return res; }
int modInverse(int e, int
phi) { for (int d = 2; d < phi;
d++) { if ((e * d) % phi == 1)
return d;
}
return -1; }
void generateKeys(int &e, int &d, int &n)
{ int p = 7919;
int q =
1009; n = p
* q;
int phi = (p - 1) * (q - 1);
for (e = 2; e < phi; e++)
{ if (__gcd(e, phi) == 1)
break; }
d = modInverse(e, phi); }
int encrypt(int m, int e, int n)
{ return power(m, e, n); }
int decrypt(int c, int d, int n)
{ return power(c, d, n); }
int main()
{ int e, d, n;
generateKeys(e, d, n);
cout << "Public Key (e, n): (" << e << ", " << n << ")\n";
cout << "Private Key (d, n): (" << d << ", " << n << ")\n";
}
```



```

n"; int M = 123;
cout << "Original Message: " << M <<
endl; int C = encrypt(M, e, n);
cout << "Encrypted Message: " << C << endl;
int decrypted = decrypt(C, d, n); cout <<
"Decrypted Message: " << decrypted <<
endl;
cout << "\nName:Raman Thapa\tRoll No:88/080\tSection:C\
t"; return 0;
}

```

Output:

```

■ PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { g++ RSAPublicprivatekey.cpp -o
publicprivatekey }; if ($?) { .\RSAPublicprivatekey }
Public Key (e, n): (7, 6177553)
Private Key (d, n): (4407043, 6177553)
Original Message: 642
Encrypted Message: 3011446
Decrypted Message: 1448114
Name:Raman Thapa      Roll No: 88/080 Section:C

```

13. Diffie-Hellman Key Exchange Implementation

C Program:

```
#include <stdio.h>

long long power(long long base, long long exp, long long
mod) { long long res = 1;
base = base % mod;
while (exp > 0) {
if (exp % 2 == 1) res = (res * base) % mod;
base = (base * base) % mod;
exp = exp / 2;
} return res; }

int main() {
long long p, g, a, b;
printf("Enter a prime number (p): ");
scanf("%lld", &p);
printf("Enter a primitive root modulo p (g):
"); scanf("%lld", &g);
printf("Enter Alice's private key (a): ");
scanf("%lld", &a);
printf("Enter Bob's private key (b): ");
scanf("%lld", &b);
long long A = power(g, a, p);
long long B = power(g, b, p);
printf("\nAlice's public key (A): %lld",
A); printf("\nBob's public key (B):
%lld", B); long long secretA =
power(B, a, p); long long secretB =
power(A, b, p);
printf("\n\nAlice's shared secret: %lld", secretA); printf("\
nBob's shared secret: %lld", secretB);
if (secretA == secretB) {
printf("\nSuccess: Shared secrets match!\n");
} else {
printf("\nError: Secrets do not match.\n"); } printf("\
nName:Raman Thapa\tRoll No:88/080\tSection:C\t");
return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc diffiehellmanex.c -o diffi
lmanex } ; if ($?) { .\diffiehellmanex }
Enter a prime number (p): 37
Enter a primitive root modulo p (g): 7
Enter Alice's private key (a): 83
Enter Bob's private key (b): 67

Alice's public key (A): 12
Bob's public key (B): 33

Alice's shared secret: 16
Bob's shared secret: 16
Success: Shared secrets match!

Name: Raman Thapa      Roll No: 88/080 Section:C
```

14. Multiplicative Inverse by Extended Euclidean

C Program:

```
#include <stdio.h>

int extended_gcd(int a, int b, int *x, int
*y) { if (a == 0) {
*x = 0; *y = 1;
return b; }
int x1, y1;
int gcd = extended_gcd(b % a, a, &x1, &y1);
*x = y1 - (b / a) * x1;
*y = x1;
return gcd;
}

int multiplicative_inverse(int a, int m)
{ int x, y;
int g = extended_gcd(a, m, &x, &y);
if(g != 1) {
return -1; }
else {
return (x % m + m) % m; }
} int main() {
int a, m;
printf("Enter the number (a): ");
scanf("%d", &a);
printf("Enter the modulus (m): ");
scanf("%d", &m);
int inverse = multiplicative_inverse(a,
m); if (inverse == -1) {
printf("Multiplicative inverse does not exist for %d modulo %d.\n", a, m);
} else {
printf("The multiplicative inverse of %d modulo %d is: %d\n", a, m, inverse); } printf("\
nName:Raman Thapa\tRoll No:88/080\tSection:C\t");
return 0; }
```

Output:

```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc Multiplicativeinverse.c -o Multiplicativein
rse } ; if ($?) { .\Multipli cativeinverse }
Enter the number (a): 67
Enter the modulus (m): 13
The multiplicative inverse of 67 modulo 13 is: 7
Name:Raman Thapa, Roll No:88/080, Section:C
```

15. Elgamal Cryptosystem:

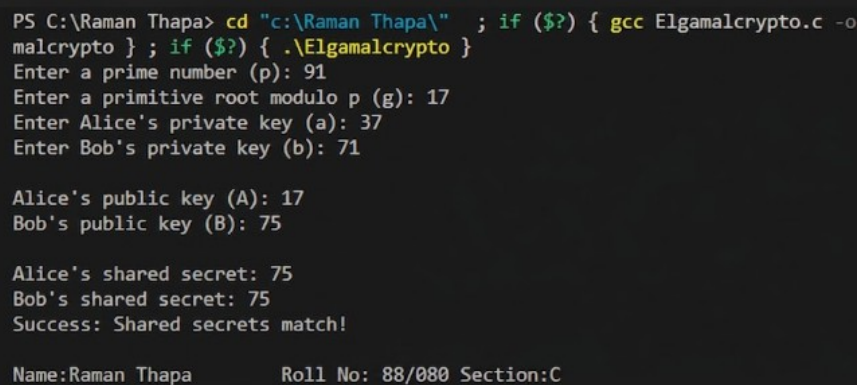
C Program:

```
#include <stdio.h>

long long power(long long base, long long exp, long long
mod){ long long res = 1;
base = base % mod;
while (exp > 0){
if (exp % 2 == 1)
res = (res * base) % mod;
base = (base * base) % mod;
exp = exp / 2; }
return res; }

int main() {
long long p, g, a, b;
printf("Enter a prime number (p): ");
scanf("%lld", &p);
printf("Enter a primitive root modulo p (g):
"); scanf("%lld", &g);
printf("Enter Alice's private key (a): ");
scanf("%lld", &a);
printf("Enter Bob's private key (b): ");
scanf("%lld", &b);
long long A = power(g, a, p);
long long B = power(g, b, p);
printf("\nAlice's public key (A): %lld",
A); printf("\nBob's public key (B):
%lld", B); long long secretA =
power(B, a, p); long long secretB =
power(A, b, p);
printf("\n\nAlice's shared secret: %lld", secretA); printf("\n
Bob's shared secret: %lld", secretB);
if (secretA == secretB)
printf("\nSuccess: Shared secrets match!\n
n"); else
printf("\nError: Secrets do not match.\n");
printf("\nName:Raman Thapa\tRoll
No:88/080\tSection:C\t"); return 0; }
```

Output:



```
PS C:\Raman Thapa> cd "c:\Raman Thapa\" ; if ($?) { gcc Elgamalcrypto.c -o
malcrypto } ; if ($?) { .\Elgamalcrypto }
Enter a prime number (p): 91
Enter a primitive root modulo p (g): 17
Enter Alice's private key (a): 37
Enter Bob's private key (b): 71

Alice's public key (A): 17
Bob's public key (B): 75

Alice's shared secret: 75
Bob's shared secret: 75
Success: Shared secrets match!

Name:Raman Thapa          Roll No: 88/080 Section:C
```